MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

Software Engineering & Product Management

#Know These Topics

UNIT 1: SOFTWARE ENGINEERING BASICS

1.1 Layered Technology

Software Engineering is a LAYERED technology. Think of it as 4 layers from bottom to top:

Layer	What It Does
Quality Focus (Bottom)	Foundation - commitment to quality in everything
Process	Framework that ties together methods, tools and people
Methods	Technical HOW-TOs: how to design, code, test
Tools (Top)	Automated support for process and methods (IDEs, testing tools)

TIP: Process is the FOUNDATION. Tools sit on top. Quality is the base of everything.

1.2 Software Process Models - The BIG Topic

Every exam has 3-4 questions on this. Know each model's USE CASE.

WATERFALL MODEL

Sequential: Requirements → Design → Code → Test → Deploy → Maintain

- Best for: Well-defined, stable requirements, regulatory/compliance projects (hospitals, government)
- Worst for: Projects where requirements CHANGE
- Key problem: No flexibility - going back is very expensive
- Exam scenario: Hospital management system with strict deadlines = WATERFALL

TIP: If question mentions 'strict deadlines', 'regulatory compliance', 'well-defined requirements' → WATERFALL

MEGAVARSHAN

AGILE / SCRUM MODEL

Iterative, sprint-based. Customer feedback drives development.

- Best for: Changing requirements, mobile apps, e-commerce with frequent updates
- Exam scenario: E-commerce with seasonal trends, food delivery app, mobile app with iterations

TIP: If question mentions 'frequent feedback', 'changing requirements', 'rapid delivery' → AGILE

SPIRAL MODEL

Risk-driven iterative model. Each loop = one phase of development.

- Best for: High-risk projects, financial institutions, large complex systems
- Key feature: Risk analysis at EVERY phase
- Exam scenario: Banking/finance project where security and risk management are TOP priorities

TIP: If question mentions 'risk management', 'security critical', 'iterative refinement' → SPIRAL

MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

PROTOTYPING MODEL

Build a quick prototype → Get feedback → Refine → Repeat

- Best for: Unclear requirements, innovative products where user reaction is unknown
- Exam scenario: E-commerce with unique never-before-seen features, cricket game app

TIP: If question mentions 'unclear requirements', 'user reaction unknown', 'feedback-driven' → PROTOTYPING

RAD (Rapid Application Development)

60-90 day cycles, multiple parallel teams, heavy component reuse

- Best for: Tight deadlines (like 90 days), when reuse is allowed
- Exam scenario: Rojer needs app in 90 days with reuse mechanism = RAD

TIP: If question mentions '90 days', 'tight deadline', 'reuse components' → RAD

INCREMENTAL MODEL

Divide project into increments. Each increment is a mini waterfall.

- Major advantage: Customer can respond to each increment
- Lower risk - early increments identify problems early

V-MODEL

Each development phase has a corresponding TEST phase.

Development Phase	Testing Phase
Requirements Gathering	Acceptance Testing → Create acceptance criteria HERE
System Design	System Testing
Architecture Design	Integration Testing
Module Design	Unit Testing
Coding	(center of V)

TIP: V-Model exam Q: During requirements phase, prepare ACCEPTANCE CRITERIA (not unit tests!)

COMPARISON TABLE - Most Important

Aspect	Waterfall	Incremental	Agile/Scrum
Approach	Linear, sequential	Iterative & incremental	Sprint-based iterative
Flexibility	Rigid - hard to change	Moderate - change in later increments	Very flexible
Risk Management	High risk - late discovery	Lower risk - early increments help	Very low - continuous feedback
Customer Involvement	Minimal - only at end	After each increment	Continuous (Sprint reviews)
Best Suited For	Stable requirements	Evolving requirements	Rapidly changing requirements
Cost of Change	Very high	Lower	Lowest

MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

UNIT 2: SCRUM (Agile Framework)

2.1 Scrum Roles

Role	Responsibilities
Product Owner	Gathers & prioritizes requirements. Represents stakeholders. Manages Product Backlog. Decides WHAT to build.
Scrum Master	Ensures Agile principles are followed. Removes blockers. Facilitates communication between PO and Developers. Does NOT manage the team.
Developers	Build and test the product. Ensure technical feasibility. Meet sprint goals.

2.2 Scrum Events

Event	Time-boxed?	Purpose
Sprint	YES (2-4 weeks)	The work period
Sprint Planning	YES	Decide what to build in this sprint
Daily Scrum (Standup)	YES (15 min)	What did I do? What will I do? Any blockers?
Sprint Review	YES	Demo to stakeholders, get feedback
Sprint Retrospective	YES	Improve team process
Product Backlog Refinement	NO (not time-boxed)	Ongoing activity to clarify backlog items

TIP: Product Backlog Refinement is NOT time-boxed !

2.3 Mid-Sprint Change Request (HOT TOPIC - asked in Set D)

If Product Owner requests major feature change mid-sprint:

- Step 1: Do NOT add it to current sprint - it disrupts the sprint goal
- Step 2: Scrum Master facilitates discussion between PO and Developers
- Step 3: Add the change to Product Backlog
- Step 4: Prioritize it for the NEXT sprint planning session
- Step 5: If change is URGENT, PO may cancel sprint and replan (extreme case)

TIP: Key rule: Protect the current sprint goal. Changes go to NEXT sprint backlog.

2.4 New Requirement in Agile

If a new requirement emerges while project is running, Assess for importance and if important to business, include it by displacing less important requirements.

UNIT 3: PROJECT MANAGEMENT & COCOMO

3.1 SDLC Phases (COCOMO Order)

Size → Effort → Duration → Cost (Remember: S-E-D-C)

TIP: Correct COCOMO order = Size, Effort, Duration, Cost

MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

3.2 COCOMO Model - MUST KNOW FORMULAS

Basic COCOMO uses 3 modes based on project type:

Category/Mode	a (effort)	b (effort exp)	c (time)	d (time exp)
Organic (simple, experienced team)	2.4	1.05	2.5	0.38
Semi-detached (medium complexity)	3.0	1.12	2.5	0.35
Embedded (complex, tight constraints)	3.6	1.20	2.5	0.32

The 4 Key Formulas:

1. EFFORT (E) = a × (KLOC)^b [person-months]
2. DEVELOPMENT TIME (T) = c × (E)^d [months]
3. AVERAGE STAFF SIZE (SS) = E / T [persons]
4. PRODUCTIVITY (P) = KLOC / E [KLOC per person-month]

Worked Example

Q1) A software project is estimated to be 800 KLOC (thousand lines of code). The project fits the organic mode category, meaning it involves a small, experienced team working on a familiar and relatively simple system. Using the COCOMO model, how would you calculate: The total effort required in person-months? The estimated time needed to complete the project in months?

E = 2.4 × (800)^1.05 ≈ 2682 person-months

T = 2.5 × (2682)^0.38 ≈ 50.2 months

Q2) A project size of 220 KLOC is being developed in the embedded mode. The development team has limited experience in this type of project, and the project schedule is strict. Calculate the effort, development time, average staff size, and productivity of the project. Assume that the values of a=3.6, b=1.20, c=2.5 and d=0.32 respectively.

E = 3.6 × (220)^1.20 ≈ 2329.22 person-months

T = 2.5 × (2329.22)^0.32 ≈ 29.88 months

SS = 2329.22 / 29.88 ≈ 77.94 people

P = 220 / 2329.22 ≈ 0.0944 KLOC/person-month

Q3) A project size of 200 KLOC is to be developed. Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the Effort, development time, average staff size, and productivity of the project.

Category	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Semi-detached chosen because: average experience + not very tight schedule

E = 3.0 × (200)^1.12 ≈ 1133.12 person-months

T = 2.5 × (1133.12)^0.35 ≈ 29.3 months

SS = 1133.12 / 29.3 ≈ 38.67 persons

P = 200 / 1133.12 ≈ 0.1765 KLOC/PM = 176 LOC/PM

TIP: Choose mode by: Organic=simple/experienced. Semi-detached=average. Embedded=complex/strict/limited experience.

3.3 Project Manager Responsibilities

- Define clear project goals aligned with client expectations
- Estimate costs and timelines, balance scope with resources
- Allocate the right resources, manage risks proactively
- Monitor progress through regular updates, adjust plans as needed
- Ensure quality and deliver product meeting client requirements

UNIT 4: REQUIREMENTS ENGINEERING

4.1 Requirement Engineering Process

The 4 steps: Elicitation → Analysis → Documentation/Specification → Validation → Management

1. Requirements Elicitation (Gathering)

- Goal: Understand stakeholder needs and define system requirements
- Techniques: Interviews, Surveys, Workshops, Observation, Prototyping
- Who defines priorities? STAKEHOLDERS (Set C Q3 answer)
- Primary goal: Understanding stakeholder needs and defining system requirements (Set C Q9)

2. Requirements Analysis

- Identify conflicts, inconsistencies, missing requirements
- Requirements conflict = different stakeholders have CONTRADICTORY requirements

3. Requirements Documentation - SRS

SRS = Software Requirements Specification

- Contains: Functional requirements, Non-functional requirements, System constraints
- Does NOT typically contain: Detailed user interface design (Set B Q9 answer: A)
- Written for: Developers and Testers (Set B Q8)
- If requirements are vague: Report to stakeholders and ask for clarification (Set D Q6)

4. Requirements Validation

Checking requirements are complete, consistent, realistic, verifiable

- Techniques: Prototyping, Requirements Reviews, Test Design, Walkthroughs
- NOT a validation technique: Software Testing (Set C Q8 - answer: D)

5. Requirements Management

- Track changes, version control, impact analysis
- When SRS needs updating: Document clearly, version control, impact analysis, get approval

MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

4.2 Types of Requirements

Type	Definition	Examples
Functional	WHAT the system does - specific behaviors	User login with username/password Heat sensor detects intrusion and alerts security Allow users to search products
Non-Functional	HOW WELL the system does it - quality attributes	System available 99.9% of time Response within 2 seconds Compliance with GDPR Security, scalability, reliability

TIP: Functional = WHAT system does. Non-Functional = HOW WELL (performance, security, usability)

4.3 Function Point Analysis (FPA)

Measures software size based on USER-FACING FUNCTIONALITY, not lines of code.

- Technology-independent (works regardless of programming language)
- Function Point = Calculated from USER REQUIREMENTS (Set C Q10 answer: A)
- Better for business apps with clear functional requirements
- 5 components: User Inputs, User Outputs, User Enquiries, User Files, External Interfaces

Function Point Calculation Formula:

UFP (Unadjusted FP) = Sum of (Count × Weight) for all 5 components

CAF (Complexity Adjustment Factor) = 0.65 + (0.01 × Sum of Fi)

FP = UFP × CAF

Worked Example:

UFP = (55×6) + (35×7) + (40×6) + (8×15) + (5×10) = 330+245+240+120+50 = 985

CAF = 0.65 + (0.01 × 56) = 0.65 + 0.56 = 1.21 [assume ΣFi = 14×4 = 56 for complex]

FP = 985 × 1.21 = 1192.85 ≈ 1193

4.4 LOC vs Function Points

Aspect	LOC (Lines of Code)	Function Points (FP)
Measures	Volume of code written	User-facing functionality
Language dependent?	YES - varies by language	NO - language independent
Better for	Coding-heavy, low-level programming	Business apps, user requirement focus
When to use	When focus is on coding effort	When user features are priority

---

UNIT 5: SCENARIO BASED

5.1 Online Sales App Requirements

Functional Requirements:

- User Authentication & Authorization (login, roles: admin/customer/seller)
- Product Catalog Management (add/update/remove products)
- Shopping Cart & Checkout (add to cart, discounts, payments)

- Order Management (tracking, cancellations, returns)
- Search & Filtering (by price, category, brand)
- Customer Support (chat, FAQs, contact)

Non-Functional Requirements:

- Performance - Fast load times
- Scalability - Handle growing users
- Security - Encryption, fraud detection
- Usability - Intuitive UI/UX
- Reliability - High availability, minimal downtime
- Compliance - GDPR, PCI-DSS

5.2 Generic Process Framework Activities

The 5 framework activities: Communication, Planning, Modeling, Construction, Deployment

TIP: Testing is NOT a framework activity - it is part of Construction!

5.3 Umbrella Activities

Umbrella activities are applied THROUGHOUT the entire software process (not just one phase).

Examples: Project management, Risk management, Quality assurance, Configuration management, Reviews

TIP: Answer = Umbrella activities (not framework activities which are sequential phases)

5.4 Requirements Elicitation with Vague Stakeholders

As Business Analyst when stakeholders have vague/conflicting ideas:

- Use open-ended questions to uncover underlying needs
- Build prototypes or mockups early to get concrete feedback
- Use Use Cases and User Stories to define real-world scenarios
- Organize workshops where ALL stakeholders share and prioritize (Set D Q9 answer: C)
- Identify conflicts early - use MoSCoW prioritization
- Document in SRS with acceptance criteria for each requirement
- Regular validation sessions with stakeholders

MEGAVARSHAN

QUICK REFERENCE CARD

Pattern Recognition - What Model?

Keywords in Question	Answer
Stable/well-defined requirements, regulatory, strict deadlines	Waterfall Model
Frequent feedback, seasonal trends, rapidly changing	Agile Model
Risk management top priority, financial institution, iterative	Spiral Model
User reaction unknown, innovative features, unclear requirements	Prototyping Model
90 days deadline, reuse mechanism allowed	RAD Model
Release in phases, improve with each increment	Incremental Model

MEGAVARSHAN. A

https://linkedin.com/in/megavarshan

https://github.com/Megavarshan

Small experienced team, familiar system	Organic Mode (COCOMO)
Average experience, normal schedule	Semi-detached Mode (COCOMO)
Complex, tight schedule, limited experience	Embedded Mode (COCOMO)

Direct Lookup

Question Theme	Answer
Applied throughout software process?	Umbrella Activities
Who defines priorities & project constraints?	Stakeholders
Major advantage of incremental model?	Customer can respond to each increment
COCOMO order?	Size → Effort → Duration → Cost
FP is calculated from?	User Requirements
NOT a validation technique?	Software Testing
NOT in SRS typically?	Detailed UI Design
SRS written for?	Developers and Testers
Requirements conflict means?	Contradictory requirements from different stakeholders
Activity NOT time-boxed in Scrum?	Product Backlog Refinement
Time-boxed delivery in Scrum?	Sprint
During Sprint, PO requests major change?	Defer to next sprint, protect current sprint goal
New requirement in Agile?	Assess importance, include by displacing less important items
NOT part of generic process framework?	Testing (it is part of Construction)
Collecting requirements from multiple stakeholders?	Organize workshop/group discussion
V-Model, requirements phase testing activity?	Creating acceptance criteria
Well-defined, unlikely to change?	Waterfall Model

MEGAVARSHAN

Highest Probability Questions:

- COCOMO calculation (Effort, Time, Staff Size, Productivity) - appeared in ALL sets!
- Compare two software models (Waterfall vs Agile/Incremental) with a scenario
- Requirements Engineering process (elicitation to management)
- Scrum roles and handling mid-sprint changes
- Function Point calculation

TIP: For COCOMO: Identify mode first (organic/semi/embedded), apply formulas in order E → T → SS → P

TIP: For model selection questions: Always justify with keywords from the scenario

GOOD LUCK! You've got this.

~ Happy Learning 😊

© Megavarshan